HACKATHON BUILD BLUEPRINT — TRACK 2 / PS 2

UniDesk AI

A Grounded Campus Helpdesk & Triage Agent

Answers only from official university documents. Cites the exact page. Asks instead of guessing. Escalates instead of hallucinating. Requires a human signature before it touches the real world.

Architecture · Tech Stack · Data Model · API Contract

Repository Structure · 24-Hour Build Plan · Demo Script

Working title. Alternatives: CampusFlow AI, CampusPilot, CampusGuardian.

Contents

1. The One-Sentence Pitch & Positioning

> **Pitch:** "Every university runs on PDFs nobody reads and an inbox nobody answers. UniDesk AI reads the PDFs, answers the student with a page citation, and when it isn't sure it doesn't guess — it files a routed ticket and waits for a human to approve anything it wants to send."

The three claims that win this

1. **Grounded.** Every answer carries document + page + section. No citation, no answer. This is enforced structurally, not by prompt politeness.
2. **Honest about uncertainty.** The agent has a numeric confidence with *reason codes*. Low confidence produces a clarification question or an escalation, never a confident guess.
3. **Governed.** No side effect (email, ticket close, record change) executes without a staff approval recorded in an immutable audit trail.

What to deliberately NOT build

The temptation is a super-app. Resist it. Judges score depth of one workflow, not breadth of six half-workflows. Explicitly out of scope for the 24 hours:

voice input | mobile app | multi-tenant auth | fine-tuning | GitHub integration | study planner | attendance prediction

Mention two of these in the pitch as "designed for, deferred by scope" — it reads as judgment, not omission.

2. Rubric-to-Feature Traceability Map

Build this table into the slide deck. It converts "we built cool stuff" into "we satisfied every stated requirement", which is what a scoring sheet rewards.

Problem statement requirement	How UniDesk satisfies it	Where it lives in the code	Visible in demo at
Grounded in trusted sources	Retrieval restricted to an admin-curated corpus of official PDFs; answers rendered only from retrieved chunks	`graph/nodes/retrieve.py`	Sources panel, right side
Transparent reasoning	Inspector panel exposes confidence, reason codes, retrieved chunks, and the decision path taken through the graph	`/api/traces/{id}`	Inspector, every turn
Handles uncertainty	Three-band confidence router: answer / clarify / escalate, each with explicit reason codes	`graph/nodes/evaluate.py`	Demo beat 3
Asks follow-up questions	Slot-filling schema per intent; missing required slots halt the graph and emit a question	`graph/slots.py`	Demo beat 2
Creates & routes tickets	Deterministic department router + priority matrix, writes to `tickets` with SLA clock	`graph/nodes/triage.py`	Admin dashboard
Takes real actions	Composio-backed Gmail draft / send and Calendar event, gated behind approval	`actions/registry.py`	Demo beat 5
Safe / responsible AI	Every side-effecting action passes the approval gateway; nothing executes on the agent's own authority	`gateway/approval.py`	Demo beat 4
Auditability	Append-only `traces` and `ticket_events`; full replay of any conversation	`db/models.py`	Trace viewer
Learns / improves	Admin corrections stored as labelled routing examples, re-injected as few-shot into the triage node	`graph/nodes/triage.py`	Slide only (stretch)

3. System Architecture

3.1 Layer view

3.2 The single most important design decision

> **The LLM never decides whether an action executes. It only proposes.** Proposal, approval, and execution are three separate rows in three separate tables, written by three different components. This is the sentence to say out loud in the pitch — it is the difference between a chatbot and a system.

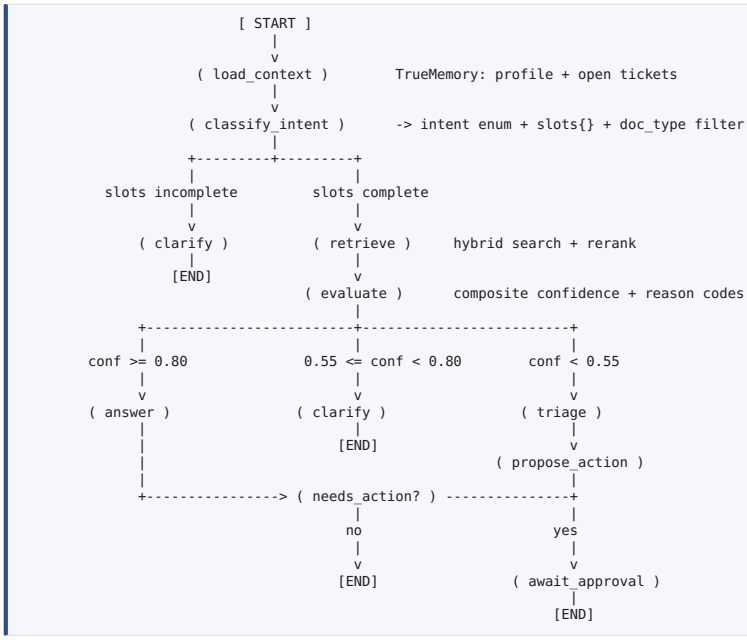
3.3 Request lifecycle (happy path, timed)

t	Stage	What happens
0ms	POST /api/chat/message	Middleware opens a trace row, sanitises input, resolves session -> student_id
15ms	load_context	TrueMemory returns profile (branch, semester, hostel block) + last 3 open tickets
250ms	classify_intent	Small LLM call returns intent enum + extracted slots as JSON
400ms	retrieve	Hybrid search, top-20, rerank to top-5, filtered by intent->doc_type map
900ms	evaluate	Composite confidence computed; router picks a branch and records the reason codes
1.2s	answer	Answer generated with forced citation markers; streamed to client via SSE
2.8s	persist	Trace closed with full state snapshot; inspector payload pushed to the right rail

Budget note: keep total under 4s for the demo. If the reranker is the bottleneck, drop it and widen top-k — the confidence model tolerates it.

4. The Reasoning Engine: LangGraph State Machine

4.1 Graph topology



4.2 State schema — write this file first

```
# graph/state.py
from typing import TypedDict, Literal, Optional, Annotated
import operator

Intent = Literal[
    "exam_policy", "attendance_query", "od_leave_request", "hostel_issue",
    "fee_scholarship", "certificate_request", "placement_query",
    "library_query", "general_info", "out_of_scope",
]

class Source(TypedDict):
    doc_id: str
    doc_title: str
    page: int
    section: str
    snippet: str
    score: float

class ActionProposal(TypedDict):
    kind: Literal["draft_email", "create_ticket", "calendar_event", "notify_slack"]
    payload: dict
    target_dept: str
    risk: Literal["low", "medium", "high"]
    requires_approval: bool

class AgentState(TypedDict):
    # --- inputs (immutable within a turn) ---
    trace_id: str
    session_id: str
    student_id: str
    query: str

    # --- context ---
    profile: dict                # branch, semester, hostel_block, attendance_pct
    open_tickets: list[dict]
    history: Annotated[list[dict], operator.add]

    # --- reasoning ---
    intent: Optional[Intent]
    slots: dict                  # filled entities
    missing_slots: list[str]
    sources: list[Source]
    confidence: float
    confidence_breakdown: dict   # component -> score
    reason_codes: list[str]

    # --- outputs ---
    route: Optional[Literal["answer", "clarify", "triage"]]
    answer_markdown: Optional[str]
    clarifying_question: Optional[str]
    ticket: Optional[dict]
    proposals: list[ActionProposal]
```

4.3 Slot schema per intent (drives the clarification behaviour)

Intent	Required slots	Clarifying question if missing
od_leave_request	leave_type, date_range, reason, proof_uploaded	"Is this Academic OD, Medical Leave, or Hostel Leave?"
exam_policy	exam_type, semester	"Which exam — mid-sem, end-sem, or a re-test?"
attendance_query	course_code or "overall", semester	"Which course, or do you want your overall percentage?"
hostel_issue	block, room_no, issue_category	"Which block and room number?"
fee_scholarship	fee_type, semester	"Is this tuition, hostel, or exam fee?"
certificate_request	certificate_type, purpose	"Bonafide, transcript, or migration certificate?"

> **Why this beats "the LLM will ask if it needs to":** it is deterministic, testable, and demoable on cue. You can guarantee the clarification fires in the demo. A prompt-based approach cannot be guaranteed, and a failed clarification in front of judges costs more than it saves in build time.

4.4 Graph assembly

```
# graph/build.py
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres import PostgresSaver
from .state import AgentState
from .nodes import (load_context, classify_intent, retrieve, evaluate,
                    answer, clarify, triage, propose_action, await_approval)

def route_after_classify(s: AgentState):
    return "clarify" if s["missing_slots"] else "retrieve"

def route_after_evaluate(s: AgentState):
    c = s["confidence"]
    if c >= 0.80: return "answer"
    if c >= 0.55: return "clarify"
    return "triage"

def route_after_answer(s: AgentState):
    return "propose_action" if s["proposals"] else END

g = StateGraph(AgentState)
for name, fn in [("load_context", load_context), ("classify_intent", classify_intent),
                 ("retrieve", retrieve), ("evaluate", evaluate), ("answer", answer),
                 ("clarify", clarify), ("triage", triage),
                 ("propose_action", propose_action), ("await_approval", await_approval)]:
    g.add_node(name, fn)

g.set_entry_point("load_context")
g.add_edge("load_context", "classify_intent")
g.add_conditional_edges("classify_intent", route_after_classify,
                        {"clarify": "clarify", "retrieve": "retrieve"})
g.add_edge("retrieve", "evaluate")
g.add_conditional_edges("evaluate", route_after_evaluate,
                        {"answer": "answer", "clarify": "clarify", "triage": "triage"})
g.add_conditional_edges("answer", route_after_answer,
                        {"propose_action": "propose_action", END: END})
g.add_edge("triage", "propose_action")
g.add_edge("propose_action", "await_approval")
g.add_edge("await_approval", END)
g.add_edge("clarify", END)

GRAPH = g.compile(checkpointer=PostgresSaver.from_conn_string(DB_URL))
```

5. Retrieval & Grounding Design

5.1 Ingestion pipeline

```
raw PDF (Supabase Storage)
       |
       v
[1] parse        PyMuPDF -> per-page text + layout blocks; keep page number
       |
       v
[2] structure    regex for "Section 4.3", "Clause 12", "Annexure B" -> section label
       |
       v
[3] chunk        700 tokens, 120 overlap, NEVER across a page boundary
       |
       v
[4] enrich       prepend a header line to every chunk:
                 "[Academic Regulations 2026 | p.18 | Sec 4.3 Medical Absence]"
       |
       v
[5] embed        bge-small-en-v1.5 (384-dim, CPU-fast, no API cost)
       |
       v
[6] index        Qdrant collection `campus_docs`, payload = full metadata
                 + BM25 sparse index in Postgres (tsvector) for hybrid
```

> **Step 4 is the trick.** Prepending the citation header into the embedded text means the retriever sees the section title as semantic signal *and* the LLM physically cannot generate the answer without the citation string sitting in its context. Grounding becomes a property of the data, not a plea in the prompt.

5.2 Query time

1. **Filter first.** Intent maps to `doc_type` (exam_policy → {exam_regulations, circulars}). Cuts the search space and kills cross-domain false positives.
2. **Hybrid.** Dense top-20 union BM25 top-20, reciprocal-rank fusion.
3. **Rerank.** `bge-reranker-base` cross-encoder down to top-5. This is where your retrieval score becomes trustworthy enough to threshold on.
4. **Recency guard.** If two chunks conflict and one has an older `effective_from`, prefer the newer and raise reason code `CONFLICTING_SOURCES`.

5.3 The answer prompt (constrained)

```
SYSTEM = """You answer questions about university policy for {branch} students in
semester {semester}.

HARD RULES
1. Use ONLY the numbered CONTEXT blocks below. You have no other knowledge of this
   university.
2. Every factual sentence must end with a marker [n] naming the context block used.
3. If the context does not contain the answer, reply with exactly: INSUFFICIENT_CONTEXT
4. Never invent a page number, section number, office name, deadline, or email address.
5. Keep it under 120 words. Plain language. No preamble.

CONTEXT
{numbered_chunks}
"""
```

Post-process: if the output contains `INSUFFICIENT_CONTEXT`, force-route to triage regardless of the numeric confidence. If any sentence lacks a `[n]` marker, strip it. This is a hard guarantee that no uncited sentence ever reaches the student.

6. The Confidence Model (and why it is not a vibe)

Most hackathon projects print a number the LLM made up. Yours is composite, decomposable, and displayed component-by-component. That single difference reads as engineering.

Component	Weight	How it is computed
Retrieval strength	0.40	Reranker score of the top chunk, min-max normalised against a calibration set
Answer support	0.30	Fraction of generated sentences whose cited chunk actually entails them (NLI model, or token-overlap fallback if time-poor)
Slot completeness	0.15	filled_required_slots / total_required_slots for the classified intent
Source authority	0.10	1.0 official regulation, 0.8 circular, 0.6 departmental FAQ, 0.3 unverified
Freshness	0.05	1.0 if effective this academic year, decays for older documents

```
# graph/confidence.py
WEIGHTS = {"retrieval": .40, "support": .30, "slots": .15, "authority": .10, "fresh": .05}

def score(parts: dict) -> tuple[float, list[str]]:
    conf = sum(WEIGHTS[k] * parts[k] for k in WEIGHTS)
    codes = []
    if parts["retrieval"] < 0.45: codes.append("NO_POLICY_MATCH")
    if parts["support"]   < 0.60: codes.append("ANSWER_NOT_ENTAILED")
    if parts["slots"]     < 1.00: codes.append("MISSING_SLOT")
    if parts["authority"] < 0.60: codes.append("LOW_AUTHORITY_SOURCE")
    if parts["fresh"]     < 0.50: codes.append("STALE_DOCUMENT")
    return round(conf, 2), codes
```

Reason codes surfaced to the user

Code	Shown to student as	Agent behaviour
NO_POLICY_MATCH	"I couldn't find a policy that covers this."	Escalate to department
MISSING_SLOT	"I need one more detail first."	Ask the slot question
ANSWER_NOT_ENTAILED	"The policy I found doesn't fully answer this."	Answer partially + escalate
CONFLICTING_SOURCES	"Two documents disagree; a human should confirm."	Show both, escalate
STALE_DOCUMENT	"This is from an older circular — verify with the office."	Answer with warning banner
OUT_OF_SCOPE	"That's outside what I have documents for."	Decline, offer ticket

> **Demo line:** "Notice it doesn't just say 61%. It says *why* it's 61% and what would raise it. That's the difference between a score and an explanation."

7. Triage: Priority Matrix & Department Routing

Do not import IT-helpdesk P1–P4 semantics. Campus urgency is driven by academic deadlines, not server uptime.

Priority	Trigger conditions	Examples	SLA	Escalates to
URGENT (red)	Deadline inside 24h, or blocks an exam/scholarship	Hall ticket missing on exam day; scholarship portal closes tonight	1 hour	Dept head + Dean office
HIGH (orange)	Deadline inside 7 days, or affects residence/safety	Attendance shortfall appeal; hostel water/electrical fault	8 hours	Department head
MEDIUM (yellow)	Administrative, time-bound but flexible	Bonafide certificate; fee receipt correction; ID card reissue	2 days	Dept queue
LOW (green)	Informational, no action pending on staff	Library timings; club registration process	5 days	Auto-close on answer

Routing table (deterministic, LLM only proposes)

Department	Owns intents	Escalation contact field
Examination Cell	exam_policy, re-test, revaluation, hall ticket	exam_officer@
Academic Office	od_leave_request, attendance_query, course registration	hod_{branch}@
Hostel Administration	hostel_issue, mess, room change, leave from hostel	warden_{block}@
Accounts & Finance	fee_scholarship, refunds, receipts	accounts@
Placement Cell	placement_query, internship NOC, eligibility	placements@
Library	library_query, fines, access	librarian@
Student Services	certificate_request, general_info, anything unrouted	studentdesk@

Implementation: a plain Python dict, not an LLM call. The LLM supplies `intent`; the dict supplies `department`. Deterministic routing is a feature you can defend under questioning.

8. Human-in-the-Loop Approval Gateway

8.1 Risk policy

Action	Risk	Approval required?	Rationale to state in the pitch
Answer with citation	low	No	Read-only, fully grounded
Create ticket	low	No	Internal record, reversible
Draft email (unsent)	low	No	Draft only; nothing leaves the system
Send email as the office	high	**Yes**	Speaks with institutional authority
Book a slot / calendar event	medium	**Yes**	Consumes a real staff resource
Close / reprioritise a ticket	medium	**Yes**	Alters the official record
Anything touching grades or fees	high	**Yes, two-person**	Irreversible and consequential

8.2 Flow

Non-negotiable invariant: the executor accepts a `proposal_id` only, never a payload from the model. It re-reads the payload from the database row that the human actually saw and approved. Otherwise a later model turn could mutate the payload between approval and execution — and a judge who works in security will ask you exactly this.

8.3 Approval record

```
{
  "approval_id": "APR-2026-0117",
  "proposal_id": "PROP-9c31",
  "trace_id":    "TRC-2026-8821",
  "action_kind": "send_email",
  "risk":        "high",
  "payload_hash":"sha256:4f2a...",       # binds decision to exact content
  "proposed_at": "2026-08-01T14:22:09Z",
  "decided_by":  "staff_42 (Academic Office)",
  "decision":    "APPROVED",
  "decided_at":  "2026-08-01T14:23:51Z",
  "note":        "Verified medical certificate attached.",
  "executed_at": "2026-08-01T14:23:52Z",
  "execution_result": {"provider":"composio.gmail","message_id":"18f..."}
}
```

9. Action Catalog (Composio)

Action	Composio tool	Payload	Gate
draft_email	GMAIL_CREATE_DRAFT	to, subject, body, ticket_ref	auto
send_email	GMAIL_SEND_EMAIL	proposal_id only	approval
book_office_slot	GOOGLECALENDAR_CREATE_EVENT	dept calendar, 15-min slot, student email	approval
notify_department	SLACK_SEND_MESSAGE	#dept channel, ticket card	auto (internal)

> **Fallback plan:** if Composio OAuth eats more than 45 minutes, ship `draft_email` as a rendered preview card in the UI plus an SMTP send behind the same approval gate. The governance story — which is what scores — is completely unaffected. Keep the Composio integration as a stretch item, not a critical-path item.

10. Memory Design

Tier	Store	Contents	Read at
Turn state	LangGraph checkpointer	Full AgentState per node transition	Resume / replay
Session	Postgres `messages`	Last N turns, verbatim	load_context
Student profile	TrueMemory	Branch, semester, hostel block, attendance %, preferred language	load_context
Episodic	Postgres `tickets`	Past tickets + resolutions for this student	load_context, triage
Institutional	Qdrant `campus_docs`	The policy corpus itself	retrieve
Corrective	Postgres `corrections`	Admin re-routings, as labelled examples	triage few-shot

The memory demo beat: student returns and says "any update on my thing from yesterday?" — agent resolves the referent from episodic memory and answers with the ticket's current status and SLA clock. Two lines of code, disproportionate impact on judges.

11. Database Schema (full DDL)

```
-- ============ CORPUS ============
create table documents (
  id            uuid primary key default gen_random_uuid(),
  title         text not null,
  doc_type      text not null,          -- exam_regulations | circular | hostel_manual | fee_policy | faq
  authority     numeric not null default 1.0,   -- feeds the confidence model
  effective_from date,
  storage_path  text not null,
  page_count    int,
  uploaded_by   text,
  created_at    timestamptz default now()
);

create table doc_chunks (
  id          bigserial primary key,
  document_id uuid references documents(id) on delete cascade,
  page        int not null,
  section     text,
  content     text not null,
  token_count int,
  tsv         tsvector generated always as (to_tsvector('english', content)) stored,
  qdrant_id   text unique               -- link to the vector store
);
create index on doc_chunks using gin(tsv);

-- ============ PEOPLE ============
create table students (
  id             text primary key,      -- roll number
  name           text not null,
  branch         text, semester int, hostel_block text,
  attendance_pct numeric, email text
);
create table staff (
  id text primary key, name text, department text not null, email text,
  can_approve_high boolean default false
);

-- ============ CONVERSATION ============
create table conversations (
  id uuid primary key default gen_random_uuid(),
  student_id text references students(id),
  started_at timestamptz default now(), status text default 'active'
);
create table messages (
  id bigserial primary key,
  conversation_id uuid references conversations(id) on delete cascade,
  role text not null,                   -- user | assistant | system
  content text not null,
  trace_id text,
  created_at timestamptz default now()
);

-- ============ AUDIT (append-only) ============
create table traces (
  id text primary key,                  -- TRC-YYYY-NNNN
  conversation_id uuid references conversations(id),
  student_id text,
  query text not null,
  intent text,
  slots jsonb default '{}',
  sources jsonb default '[]',           -- [{doc_id,title,page,section,score}]
  confidence numeric,
  confidence_breakdown jsonb,
  reason_codes text[],
  route text,                           -- answer | clarify | triage
  answer text,
  latency_ms int,
  model text,
  created_at timestamptz default now()
);
revoke update, delete on traces from public;

-- ============ TICKETS ============
create table tickets (
  id text primary key,                  -- TICK-YYYY-NNNN
  student_id text references students(id),
  trace_id text references traces(id),
  title text not null, body text,
  department text not null,
  priority text not null check (priority in ('URGENT','HIGH','MEDIUM','LOW')),
  status text not null default 'OPEN'
        check (status in ('OPEN','IN_PROGRESS','AWAITING_STUDENT','RESOLVED','CLOSED')),
  sla_due_at timestamptz,
  assigned_to text references staff(id),
  created_at timestamptz default now(),
  resolved_at timestamptz
);
create table ticket_events (
  id bigserial primary key,
  ticket_id text references tickets(id) on delete cascade,
  event_type text not null,             -- CREATED|ROUTED|REPRIORITISED|COMMENTED|RESOLVED
  actor text not null,                  -- 'agent' | staff_id
  detail jsonb, created_at timestamptz default now()
);
```

```
-- ============ GOVERNANCE ============
create table approvals (
  id text primary key,                  -- APR-YYYY-NNNN
  proposal_id text unique not null,
  trace_id text references traces(id),
  ticket_id text references tickets(id),
  action_kind text not null,
  risk text not null check (risk in ('low','medium','high')),
  payload jsonb not null,
  payload_hash text not null,
  status text not null default 'PENDING'
        check (status in ('PENDING','APPROVED','REJECTED','EXECUTED','FAILED')),
  proposed_at timestamptz default now(),
  decided_by text references staff(id),
  decided_at timestamptz, note text,
  executed_at timestamptz, execution_result jsonb
);

-- ============ LEARNING LOOP ============
create table corrections (
  id bigserial primary key,
  trace_id text references traces(id),
  field text not null,                  -- department | priority | intent
  agent_value text, corrected_value text,
  corrected_by text references staff(id),
  created_at timestamptz default now()
);
```

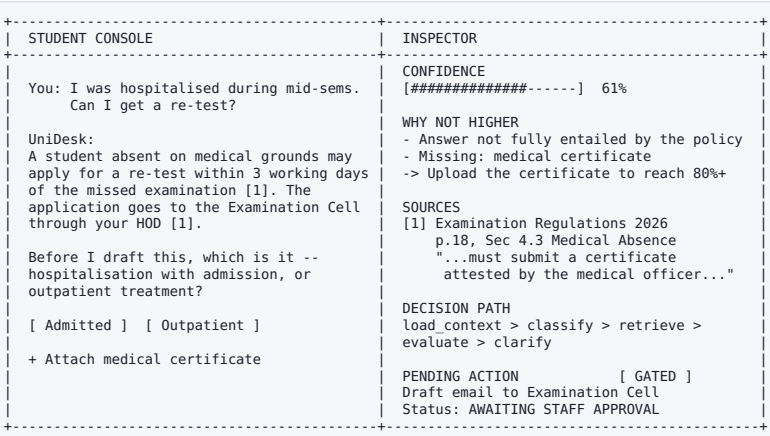
12. API Contract

Verb	Path	Body / params	Returns
POST	/api/chat/message	{session_id, student_id, text}	SSE stream: tokens, then a final `inspector` event
GET	/api/chat/{session_id}	—	Full message history + trace ids
POST	/api/chat/{session_id}/upload	multipart (medical certificate etc.)	{attachment_id} → fills a slot
POST	/api/admin/documents	multipart PDF + doc_type, authority, effective_from	{document_id, chunks_indexed}
GET	/api/tickets	?dept=&priority=&status=&overdue=true	Ticket list with SLA countdown
PATCH	/api/tickets/{id}	{status?, priority?, department?, note?}	Updated ticket; writes ticket_event + correction
GET	/api/approvals?status=PENDING	—	Approval queue with rendered payload preview
POST	/api/approvals/{id}/decide	{decision: APPROVED|REJECTED, note}	Executes on approve; returns execution_result
GET	/api/traces/{trace_id}	—	Full reasoning replay for the inspector / trace viewer
WS	/ws/admin	—	Live push: new ticket, new approval, SLA breach

Inspector payload (the object the right rail renders)

```
{
  "trace_id": "TRC-2026-8821",
  "confidence": 0.61,
  "breakdown": {"retrieval":0.72,"support":0.48,"slots":0.50,"authority":1.0,"fresh":1.0},
  "reason_codes": ["ANSWER_NOT_ENTAILED","MISSING_SLOT"],
  "next_step_hint": "Upload your medical certificate to raise confidence above 80%.",
  "route": "clarify",
  "path": ["load_context","classify_intent","retrieve","evaluate","clarify"],
  "intent": "od_leave_request",
  "slots": {"leave_type":"medical","date_range":"2026-07-14..2026-07-16",
            "proof_uploaded": false},
  "sources": [
    {"title":"Examination Regulations 2026","page":18,"section":"4.3 Medical Absence",
     "score":0.72,"snippet":"A student absent on medical grounds must submit..."}
  ],
  "latency_ms": 2810,
  "pending_approval": {"id":"APR-2026-0117","action":"send_email","risk":"high"}
}
```

13. Frontend: The Transparency Panel



Component checklist

- `ConfidenceMeter` — animated bar, colour-banded at 0.55 / 0.80. Show the number.
- `ReasonCodeList` — human-readable strings, never raw enum names, plus the "next step" hint.
- `SourceCard` — title, page, section, snippet; click opens the PDF at that page in a viewer.
- `DecisionPath` — the actual node sequence, rendered as breadcrumbs. Cheap to build, unusually persuasive.
- `ApprovalBadge` — GATED / APPROVED / EXECUTED with timestamps.
- `QuickReplyChips` — clarification options as buttons. Makes the demo fast and unmissable.

> **Build the source-card PDF deep-link.** Clicking a citation and watching the actual regulation PDF open on page 18 with the clause highlighted is the single highest-impact 30 minutes of frontend work in this project. It converts "it says it has a source" into "I just saw the source."

14. Repository Structure

```
unidesk-ai/
├── README.md                     # architecture diagram + 3-command quickstart
├── docker-compose.yml            # postgres, qdrant, api, web
├── .env.example
│
├── apps/
│   └── web/                      # Next.js 14 (adapt PaperBuddy shell)
│       ├── app/
│       │   ├── chat/page.tsx
│       │   ├── admin/
│       │   │   ├── tickets/page.tsx
│       │   │   ├── approvals/page.tsx
│       │   │   └── traces/[id]/page.tsx
│       │   └── layout.tsx
│       ├── components/
│       │   ├── chat/{MessageList,Composer,QuickReplyChips}.tsx
│       │   ├── inspector/{ConfidenceMeter,ReasonCodeList,SourceCard,DecisionPath}.tsx
│       │   └── admin/{TicketTable,ApprovalCard,SlaBadge}.tsx
│       └── lib/{api.ts,sse.ts,ws.ts}
│
├── services/
│   └── api/
│       ├── main.py               # FastAPI app + middleware
│       ├── config.py             # pydantic-settings
│       ├── deps.py
│       ├── routers/
│       │   ├── chat.py  tickets.py  approvals.py  documents.py  traces.py
│       ├── graph/
│       │   ├── build.py          # StateGraph assembly
│       │   ├── state.py          # AgentState TypedDict
│       │   ├── slots.py          # intent -> required slots
│       │   ├── confidence.py     # composite scorer + reason codes
│       │   ├── routing.py        # intent -> department, priority rules
│       │   └── nodes/
│       │       ├── load_context.py  classify_intent.py  retrieve.py
│       │       ├── evaluate.py      answer.py           clarify.py
│       │       ├── triage.py        propose_action.py   await_approval.py
│       ├── rag/
│       │   ├── ingest.py         # parse -> chunk -> enrich -> embed -> index
│       │   ├── search.py         # hybrid + rerank
│       │   └── prompts.py
│       ├── gateway/
│       │   ├── policy.py         # risk classification rules
│       │   └── approval.py       # propose / decide / execute
│       ├── actions/
│       │   ├── registry.py       # action_kind -> handler
│       │   └── composio_client.py
│       ├── memory/truememory.py
│       ├── db/{models.py,session.py,migrations/}
│       └── tests/
│           ├── test_confidence.py      # golden thresholds
│           ├── test_routing.py         # every intent lands in a department
│           └── test_gateway.py         # high-risk NEVER auto-executes
│
├── data/
│   ├── seed_pdfs/                # 6-8 official documents
│   ├── seed_students.sql
│   └── golden_questions.yaml     # 25 Q&A pairs w/ expected doc+page
│
└── scripts/
    ├── ingest_all.py
    ├── seed_db.py
    └── eval_retrieval.py         # recall@5 + citation accuracy
```

15. Environment & Configuration

```
# .env.example
APP_ENV=dev
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/unidesk
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=campus_docs

LLM_PROVIDER=gemini             # gemini | ollama | openai
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.0-flash
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.1:8b        # offline fallback -- pre-pull before the venue

EMBED_MODEL=BAAI/bge-small-en-v1.5
RERANK_MODEL=BAAI/bge-reranker-base
RERANK_ENABLED=true

CONF_ANSWER_THRESHOLD=0.80
CONF_CLARIFY_THRESHOLD=0.55
AUTO_APPROVE_MAX_RISK=low

COMPOSIO_API_KEY=
COMPOSIO_ENTITY_ID=unidesk-demo
SMTP_HOST=                       # fallback path if Composio OAuth stalls
```

Do this the night before, not at the venue: pull the Ollama model, download the embedding and reranker weights into the repo cache, and run the full ingest once. Venue wifi is the most reliable cause of hackathon failure. Commit a `models/` cache or carry it on a USB drive.

16. Tech Stack Decision Table

Layer	Choice	Why this one	Fallback if it burns >60 min
Frontend	Next.js 14 + Tailwind + shadcn/ui	Reuse the PaperBuddy shell; App Router handles SSE cleanly	Single-page Vite + React
API	FastAPI (async)	Native async, SSE, Pydantic validation matching the state schema	— (no substitute needed)
Orchestration	LangGraph	Explicit state machine; conditional edges are the demo's backbone	Hand-rolled async state machine, same node signatures
LLM	Gemini 2.0 Flash	Fast, cheap, strong JSON mode for intent + slot extraction	Ollama llama3.1:8b, fully offline
Embeddings	bge-small-en-v1.5	384-dim, runs on CPU, no API dependency or quota risk	—
Vector store	Qdrant (docker)	Payload filtering by doc_type is first-class and fast	pgvector in the Postgres you already run
Live ingest	Pathway	Optional: live re-index when admin drops a new circular	Cut it. Manual upload endpoint is enough.
Memory	TrueMemory	Local profile store; zero network dependency	A `students` table join
Approval	clawvisor policy layer	Named, external governance component reads as deliberate	Your own `gateway/` module — same schema
Actions	Composio	One integration surface for Gmail + Calendar + Slack	SMTP draft preview behind the same gate
Database	Supabase / Postgres	Storage for PDFs, Realtime for the admin queue, plain SQL	Local Postgres + local file storage
Deploy	Vercel (web) + Railway/Render (api)	Fast, free tier, public URL for judges to try	ngrok from a laptop — but have this ready

The stack discipline rule: every tool in this table must be visible in the two-minute demo or defensible in one sentence. If neither is true, delete it. Pathway and ratel are the two most likely deletions — that is fine, and saying "we cut it for scope" scores better than a half-wired integration.

17. The 24-Hour Build Plan

Pre-hackathon (do this before you arrive — it is legal, it is expected, and it decides the outcome): repo scaffolded, docker-compose running, DB migrated, models cached locally, seed PDFs collected and ingested, PaperBuddy shell stripped down to two routes. Arrive with a system that boots. Hour 0 should be feature work, not `npm install`.

Window	Milestone (must be demoable at the end of it)	Detail
H+0 – H+2	**Vertical slice**: question in, cited answer out	Hardcode intent, skip confidence. One route, one retrieval call, one LLM call, citations rendering. Nothing else matters until this works.
H+2 – H+4	Ingestion pipeline over all seed PDFs	parse → chunk → enrich header → embed → index. Run `eval_retrieval.py`; target recall@5 ≥ 0.85 on the golden set.
H+4 – H+6	Real LangGraph with all nodes wired	classify_intent + slots + evaluate + three-way router. Confidence can be crude here; the *branching* must be real.
H+6 – H+8	Inspector panel live	Confidence meter, reason codes, source cards, decision path. This is the scoring surface — build it early, not last.
H+8 – H+10	Triage + tickets + admin queue	Routing dict, priority rules, SLA clock, ticket table with filters. Seed 8 fake tickets so the dashboard never looks empty.
H+10 – H+12	**Approval gateway end to end**	propose → PENDING row → WS push → approval card → approve → execute → result written back. This is your differentiator; give it real time.
H+12 – H+13	Hard freeze on new features	Whatever is not working now gets cut, not fixed. Write the cut list down; it becomes your "future work" slide.
H+13 – H+15	Composio action + email draft	The only post-freeze exception. If OAuth fights you past 45 minutes, switch to the SMTP fallback immediately and move on.
H+15 – H+17	Calibration pass	Run the 25 golden questions. Tune thresholds so each demo question lands in its intended band. Record the numbers — you will be asked.
H+17 – H+19	Polish + seed the demo state	Empty states, loading skeletons, error toasts, favicon, product name in the header. Pre-load the exact student account you will demo with.
H+19 – H+21	Deploy + record a backup video	Public URL live. Then screen-record the full demo. If anything breaks on stage you play the video and keep talking. Non-negotiable.
H+21 – H+23	Deck + rehearsal	10 slides max. Rehearse the two-minute demo four times, out loud, with a timer, on the venue wifi.
H+23 – H+24	Freeze, sleep, commit	Tag the release. Do not push after this. Every hackathon has a team that breaks main at minute 23.

18. Team Split & Interface Contracts

Role	Owns	Hands off
Agent / Orchestration	LangGraph nodes, state schema, confidence model, routing rules, prompts	The `AgentState` TypedDict and the inspector JSON, both frozen at H+1
Retrieval / Data	Ingestion pipeline, Qdrant, hybrid search, reranker, golden question set, eval script	A `search(query, doc_types, k) -> list[Source]` function, frozen at H+1
Frontend	Both consoles, inspector panel, admin queue, approval inbox, SSE/WS plumbing	Consumes only the frozen inspector JSON — mock it at H+0 and never block on the backend
Platform / Demo	DB schema, migrations, gateway + executor, Composio, deploy, seed data, deck, backup video	The `approvals` table contract, frozen at H+1

The contract-first rule. In the first hour, all four of you agree on three JSON shapes: `AgentState`, the inspector payload, and the approval record. Write them into the repo as literal files with example data. After that, everyone builds against the shape, not against each other's progress. This is the single biggest determinant of whether four people go 4x or 1.2x.

19. Seed Data Plan

Your corpus quality caps your demo quality. Collect these before the event — most are already public on your university site.

Document	doc_type	authority	Unlocks which demo question
Academic Regulations / Handbook	exam_regulations	1.0	Re-test, revaluation, attendance shortfall
Examination Manual	exam_regulations	1.0	Medical absence, hall ticket rules
Attendance & OD Policy	circular	0.8	OD application, condonation
Hostel Rules & Mess Manual	hostel_manual	1.0	Room change, leave, complaints
Fee Structure & Refund Policy	fee_policy	1.0	Refund window, late fee
Scholarship Guidelines	circular	0.8	Eligibility, deadline (urgent-priority demo)
Placement Policy	circular	0.8	Eligibility, NOC, one-offer rule
An *intentionally superseded* 2023 circular	circular	0.6	**The conflict demo** — triggers CONFLICTING_SOURCES + STALE_DOCUMENT

Deliberately include the stale document. Most teams curate a clean corpus so nothing goes wrong. Including a superseded circular and showing the agent detect the conflict, prefer the newer rule, and flag it for human confirmation is a 20-second moment that no other team will have.

golden_questions.yaml (build 25; here is the shape)

```
- q: "I was hospitalised during mid-sems, can I get a re-test?"
  expect_intent: exam_policy
  expect_doc: "Examination Regulations 2026"
  expect_page: 18
  expect_band: clarify          # missing certificate slot
- q: "What is the last date to pay the hostel fee?"
  expect_intent: fee_scholarship
  expect_band: answer
- q: "Can I switch my hostel room because my roommate snores?"
  expect_intent: hostel_issue
  expect_band: triage           # no policy covers it -> NO_POLICY_MATCH
- q: "Who won the inter-college cricket final?"
  expect_intent: out_of_scope
  expect_band: triage           # must decline cleanly
```

Run these as a pytest suite. "We have an evaluation set and here are our numbers" is a sentence almost no hackathon team can say, and it lands hard with technical judges.

20. Demo Script (2 minutes, word for word)

Time	Beat	What you say / do
0:00	The problem	"Every university answer already exists — in a 90-page PDF nobody reads. So students email offices, and offices drown. UniDesk reads the PDFs."
0:15	Grounded answer	Ask: *"What's the attendance requirement to sit for end-sems?"* Answer streams in. **Click the citation** — the actual regulation PDF opens on that page. "It didn't recall this. It read it."
0:40	Refuses to guess	Ask: *"I was hospitalised during mid-sems, can I get a re-test?"* Confidence drops to 61%. Point at the panel: "It tells you *why* — the answer isn't fully entailed and the certificate is missing. So it asks instead of guessing."
1:00	Clarify + upload	Click **Admitted**, attach the certificate. Confidence climbs past 80%. Agent produces the exact procedure and offers to draft the application.
1:20	**The governance beat**	Accept the draft. It does **not** send. "The agent proposed. It cannot execute." Switch to the staff console — the approval is already sitting there, live.
1:35	Human approves	Approve as staff. Email sends, ticket appears in the Examination Cell queue as HIGH with an SLA clock running.
1:50	The audit close	Open the trace viewer. "Every retrieved document, every confidence component, every approval — immutable. If a university adopts this, that's what their compliance office will ask for first."

Rehearsal rule: the person talking is never the person clicking. The presenter faces the judges the whole time. One teammate drives the screen on cue. Practise this — it is the visible difference between a team that built something and a team that rehearsed something.

21. Risk Register & Panic Buttons

Risk	Likelihood	Mitigation / panic button
Venue wifi collapses	High	Ollama model pre-pulled; embeddings local; `LLM_PROVIDER=ollama` is a one-line switch. Test the switch at H+16, not at demo time.
Composio OAuth loop	High	Hard 45-minute timebox, then SMTP fallback behind the same gate. Governance story unchanged.
Retrieval returns the wrong section	Medium	doc_type filtering + reranker. If still wrong, hardcode a doc_type hint for the demo intents — and say you did, framed as intent-scoped retrieval.
LLM answers without citation markers	Medium	Post-processor strips uncited sentences. If everything is stripped → INSUFFICIENT_CONTEXT → escalate. Fails safe, never embarrassing.
Latency over 6s on stage	Medium	Disable the reranker via env flag; pre-warm the model with a dummy call 60s before demo.
Demo machine dies	Low	Backup video recorded at H+21. Deployed public URL as a second path.
Someone breaks main at hour 23	Medium	Tag `v1-demo` at H+21. Demo from the tag, not from main.
Scope creep from a teammate's good idea	Very High	The H+12 freeze is a rule, not a suggestion. New ideas go on the "future work" slide, which is itself a scoring asset.

22. Judging Q&A Prep

Question you will be asked	Answer
"How do you know it isn't hallucinating?"	"Three layers. Retrieval is filtered to an admin-curated corpus. The generator is structurally forced to emit a citation marker per sentence, and uncited sentences are stripped before display. Then an entailment check scores whether the cited chunk actually supports the sentence — that score is 30% of the confidence number you can see on screen."
"Where does the confidence number come from?"	"It's a weighted composite of five measurable components, all displayed in the panel. No LLM self-reports its own confidence anywhere in this system."
"What if the agent sends a wrong email?"	"It structurally cannot send anything. It writes a proposal row. A staff member sees the exact rendered payload and approves it. The executor takes a proposal ID and re-reads the approved payload from the database — the model has no path to the send call."
"How is this different from ChatGPT with the PDFs uploaded?"	"Three things ChatGPT can't do here: it can't refuse on a calibrated threshold and route to the right department instead; it can't hold an action pending a named staff member's approval; and it can't give a compliance office an immutable trace of why every answer was given. This is a workflow system, not a chat window."
"Does it scale to a real university?"	"Retrieval is a single Qdrant collection with payload filters — it scales with documents, not with students. The stateful parts are Postgres rows. The realistic bottleneck is corpus curation, which is why document upload is an admin-authenticated route with an authority score, not an open ingest."
"What did you not build, and why?"	Name two things from the cut list with the reason. This answer, delivered calmly, is worth more than one more half-working feature.
"What's your evaluation?"	"25 golden questions with expected document, page, and confidence band. Recall@5 is X, citation accuracy is Y, band accuracy is Z." Have the real numbers on a slide.

23. Stretch Goals, Ranked by Return

1. **Citation deep-link into the PDF viewer** — highest impact per hour of any item in this document. Do it even if it means cutting something else.
2. **The learning loop, visibly.** Admin re-routes a ticket; the correction is stored; the next similar query routes correctly. If you can show this in 20 seconds, show it.
3. **SLA breach escalation** — a background job that bumps priority and notifies the department head when a ticket ages past its SLA. Small code, strong "real system" signal.
4. **Two-person approval** for the highest-risk class. One extra column, one extra check, and a very good sentence in the pitch.
5. **Multilingual answers** (Hindi / regional). Retrieve in English, answer in the student's language, keep the citation in the source language. High delight, moderate cost.
6. **Live re-index via Pathway** — drop a new circular into a folder, watch it become answerable within seconds. Genuinely impressive, but only if the core is completely done.

Final note

> Almost every team at this hackathon will build a chatbot over a PDF. The retrieval will work. The answers will be fine. They will all look the same by the fourth demo of the afternoon.
>
> What you are building is a chatbot over a PDF **plus three things nobody else will have**: a confidence number that decomposes into reasons, an action that refuses to execute without a human signature, and an audit trail that a compliance office would accept. Those three things are cheap to build and impossible to fake in a demo.
>
> Protect them. Cut everything else before you cut those.